

JAVA WEB

Unit 4: Servlet



Introduction

- Giới thiệu Servlet
 - Luồng xử lý trong Web Servlet
 - Cấu trúc của một lớp Servlet
 - Vòng đời của Servlet
 - Vấn đề đa luồng
 - Gỡ rối chương trình Servlet
- Đọc dữ liệu từ Form HTML
- Lọc kí tự đặc biệt
- Cộng tác giữa các Servlets

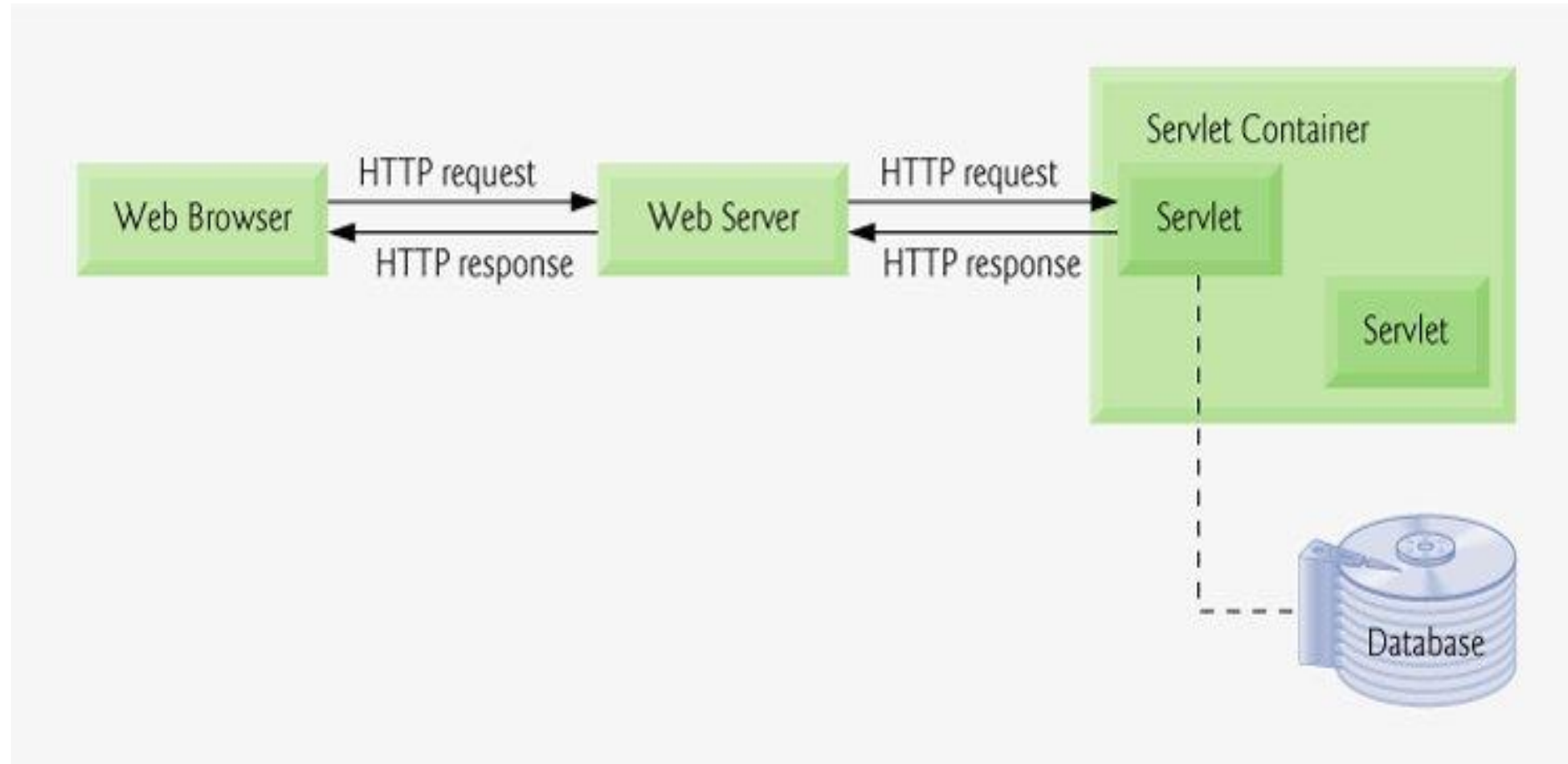


Giới thiệu Servlet

- ✓ Luồng xử lý trong Servlet
- ✓ Cấu trúc cơ bản của một Servlet
- ✓ Phát triển ứng dụng Web trên IntelliJ, NetBeans với Servlet
- ✓ Vòng đời của Servlet
- ✓ Vấn đề đa luồng trong thực thi một Servlet
- ✓ Gỡ rối chương trình Servlet

UVINA

Luồng xử lý trong Servlet



Cấu trúc cơ bản của Servlet

Lớp Servlet kế thừa từ lớp **HttpServlet**; các phương thức **doGet**, **doPost** thực hiện hành động tương ứng với các phương thức Get, Post của request.

```
import java.io.*;
import javax.servlet.*; import javax.servlet.http.*;
public class ServletTemplate extends HttpServlet {
    public void doGet (HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Use "request" to read incoming
        // Use "response" to write ....
        PrintWriter out = response.getWriter();
        // Use "out" to send content to browser.
    }
}
```

Các phương thức cơ bản của Servlet

Phương thức khởi tạo:

Thực hiện **duy nhất** trong **lần gọi đầu tiên** và không được thực hiện lại khi có tiếp các request từ client. Có 2 nạp chồng sau:

- ✓ **public void init() throws ServletException**
- ✓ **Public void init(ServletConfig config) throws Servlet Exception**

(Tham số **config cho phép** đọc các thông tin liên quan đến việc khởi tạo như thông tin xác thực: username, password) ...

Các phương thức cơ bản của Servlet (tt)

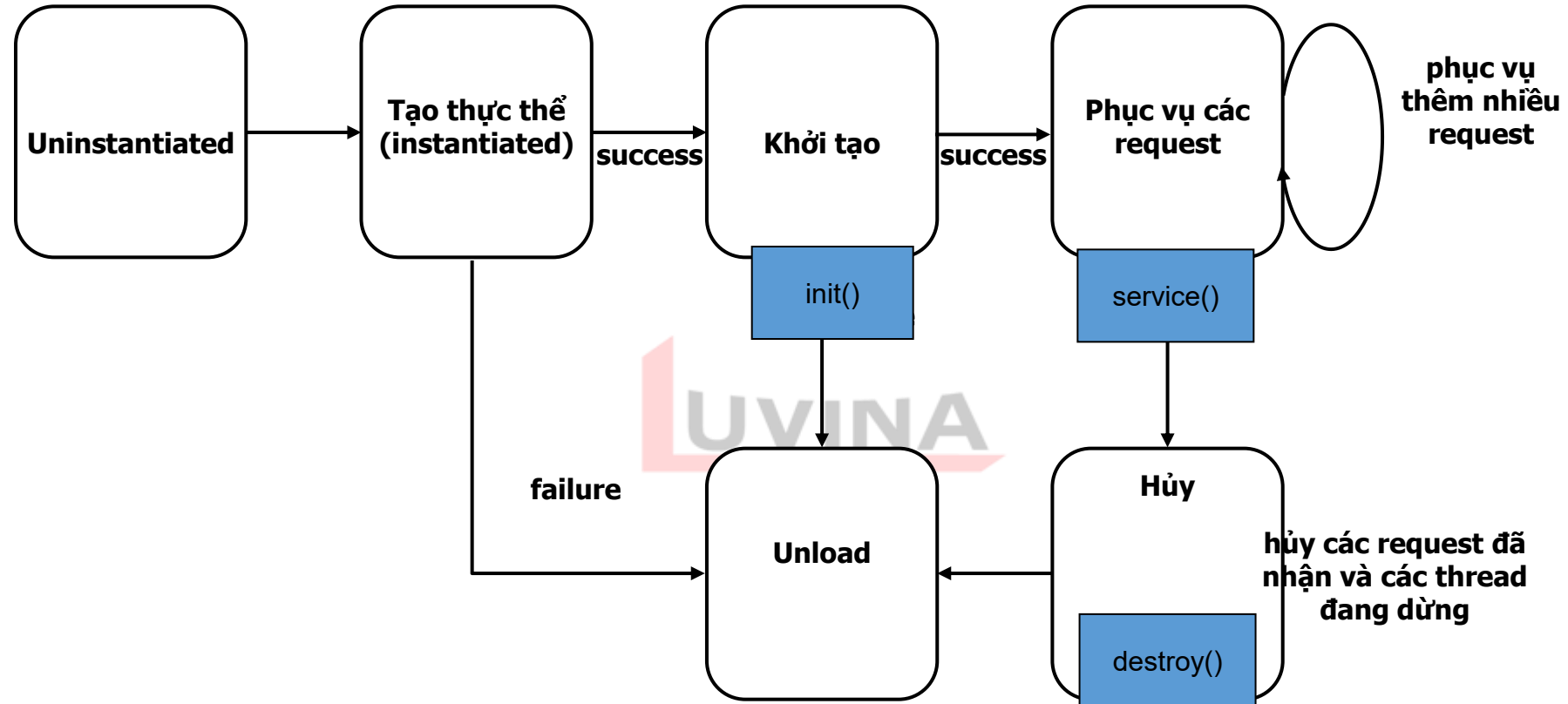
Phương thức **service()**:

- ✓ Được gọi khi có **request** từ client thông qua luồng threads. Phương thức service kiểm tra loại http request (GET, POST, ...) để gọi các phương thức tương ứng như **doXXX** ...

- ✓ Khai báo phương thức service () như sau:

```
public void service (HttpServletRequest request, HttpServletResponse response)  
    throws ServletException, IOException
```

Vòng đời của Servlet



Cách tạo lớp kiểu Servlet

Step 1: import các gói hỗ trợ trong xây dựng và thực thi servlet

➤ `javax.servlet.*`

➤ `javax.servlet.http.*`

Step 2: Tạo java class extends **HttpServlet** (loại public)

Step 3: Implement các phương thức `init`, `destroy` (nếu cần) và `doXXX` cùng với phương thức của đối tượng `PrintWriter` để kết xuất lên trang Web.

Step 4: Thực hiện tạo gói module Web (jar,war), deploy và thực thi ứng dụng

ServletHello.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class Hello extends HttpServlet {
    public void doGet (HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out=response.getWriter();
        out.println("<html><head><title>Hello Servlet</title></head><body>");

        out.println("<h1>Hello Servlet</h1>");

        out.println("</body></html>");
    }
    public void doPost (HttpServletRequest request, HttpServletResponse response ) throws ServletException, IOException {
        doGet (request, response);
    }
}
```

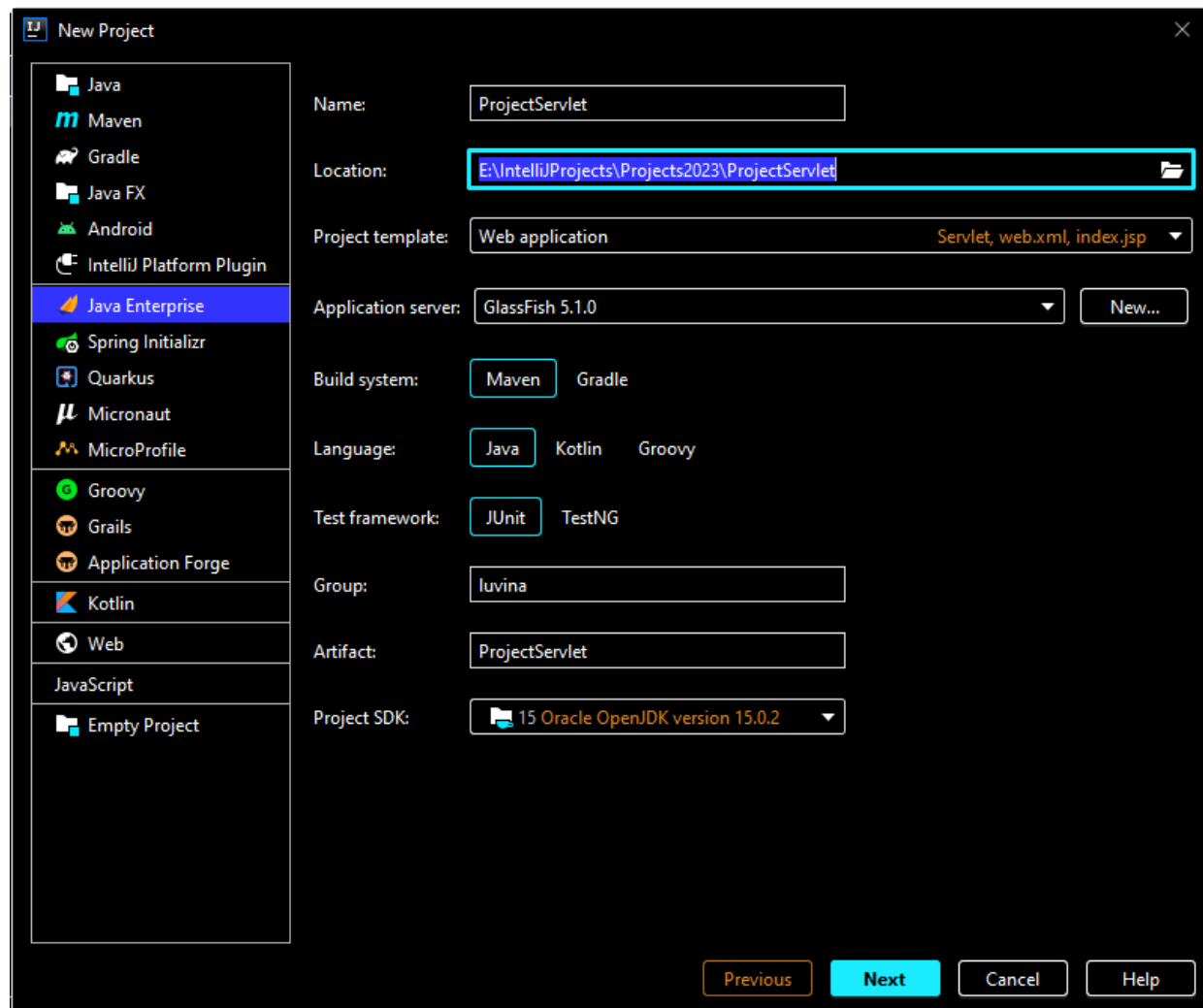
Phát triển ứng dụng Web

Install JDK **15** (hoặc mới hơn)(tải trên trang web của SUN).

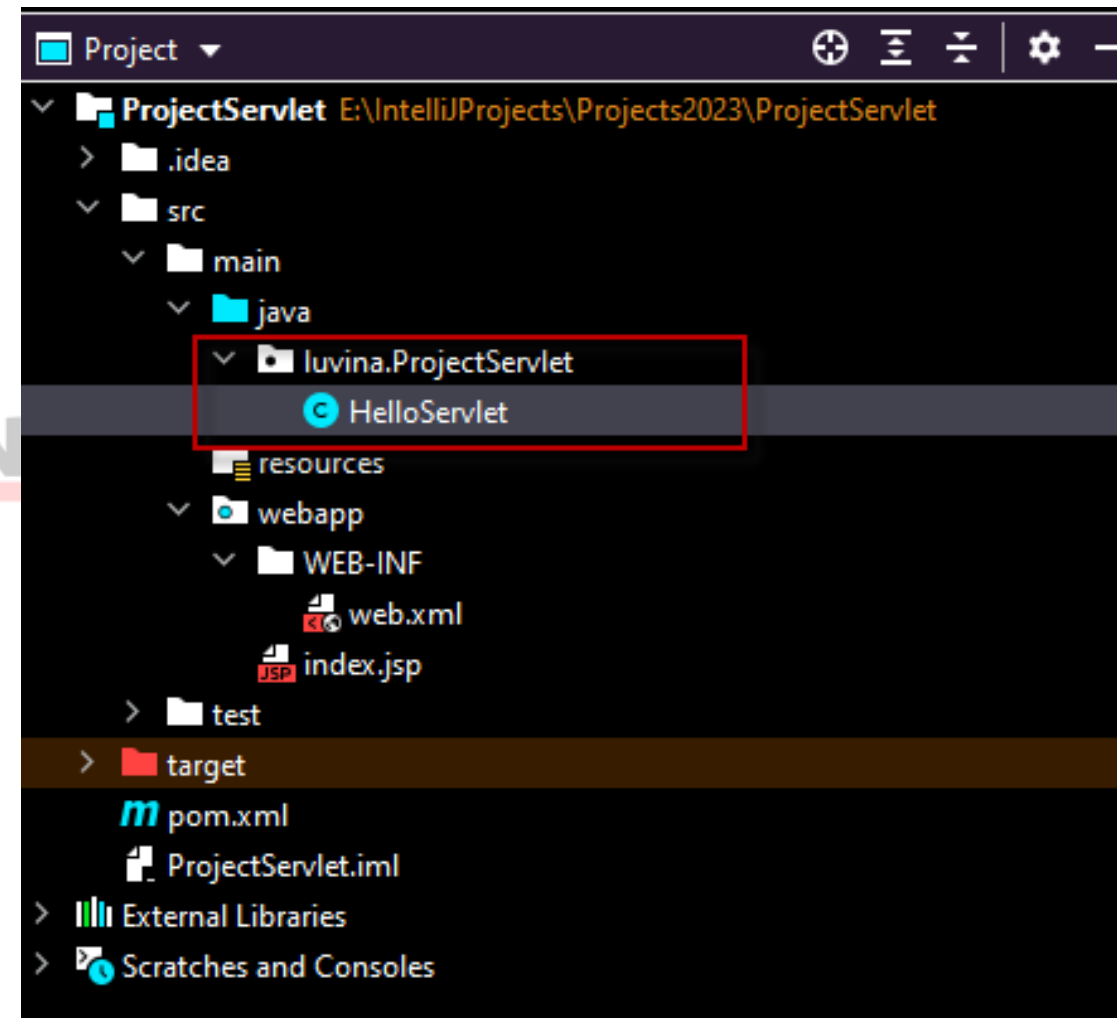
Install Web Application Server: **Glassfish**; **Tomcat**.

Sử dụng công cụ soạn thảo chương trình như **Netbeans**, **Eclipse**, **MyEclipse**, **IntelliJ**, ...

Tạo ứng dụng Web trên IntelliJ

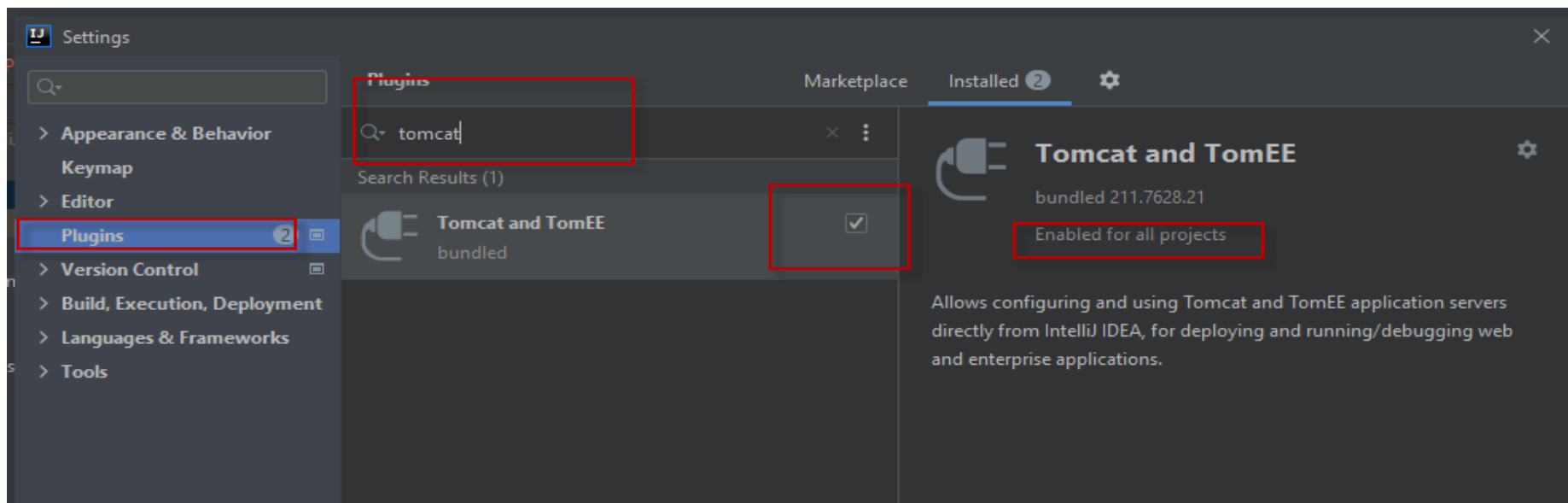
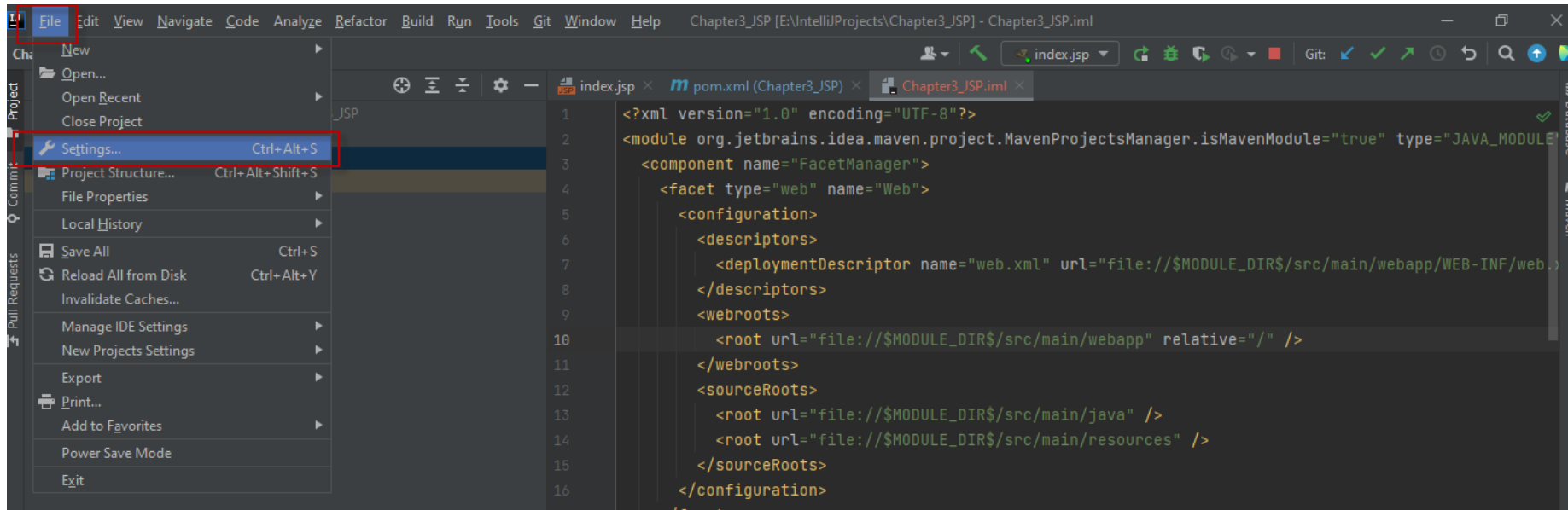


Cấu trúc của Project

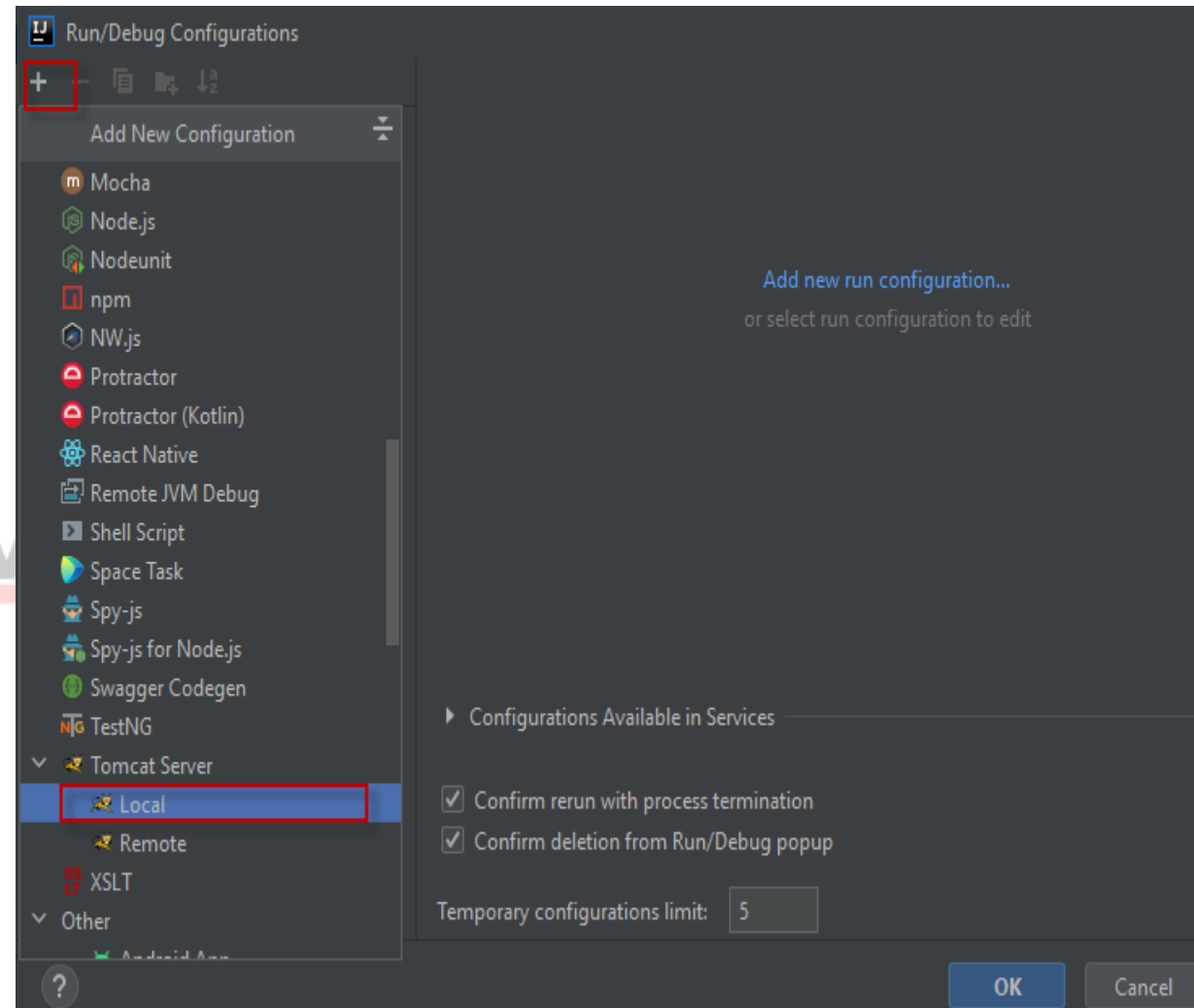
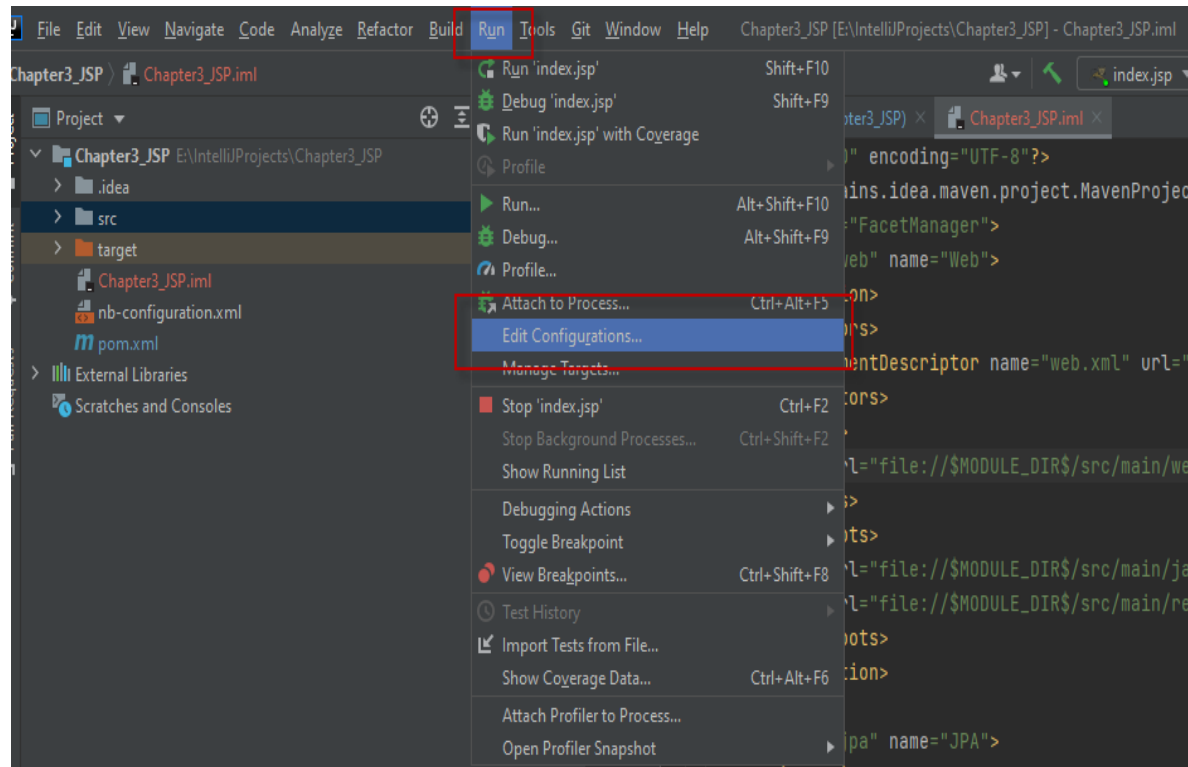


Cấu hình Web Server

13

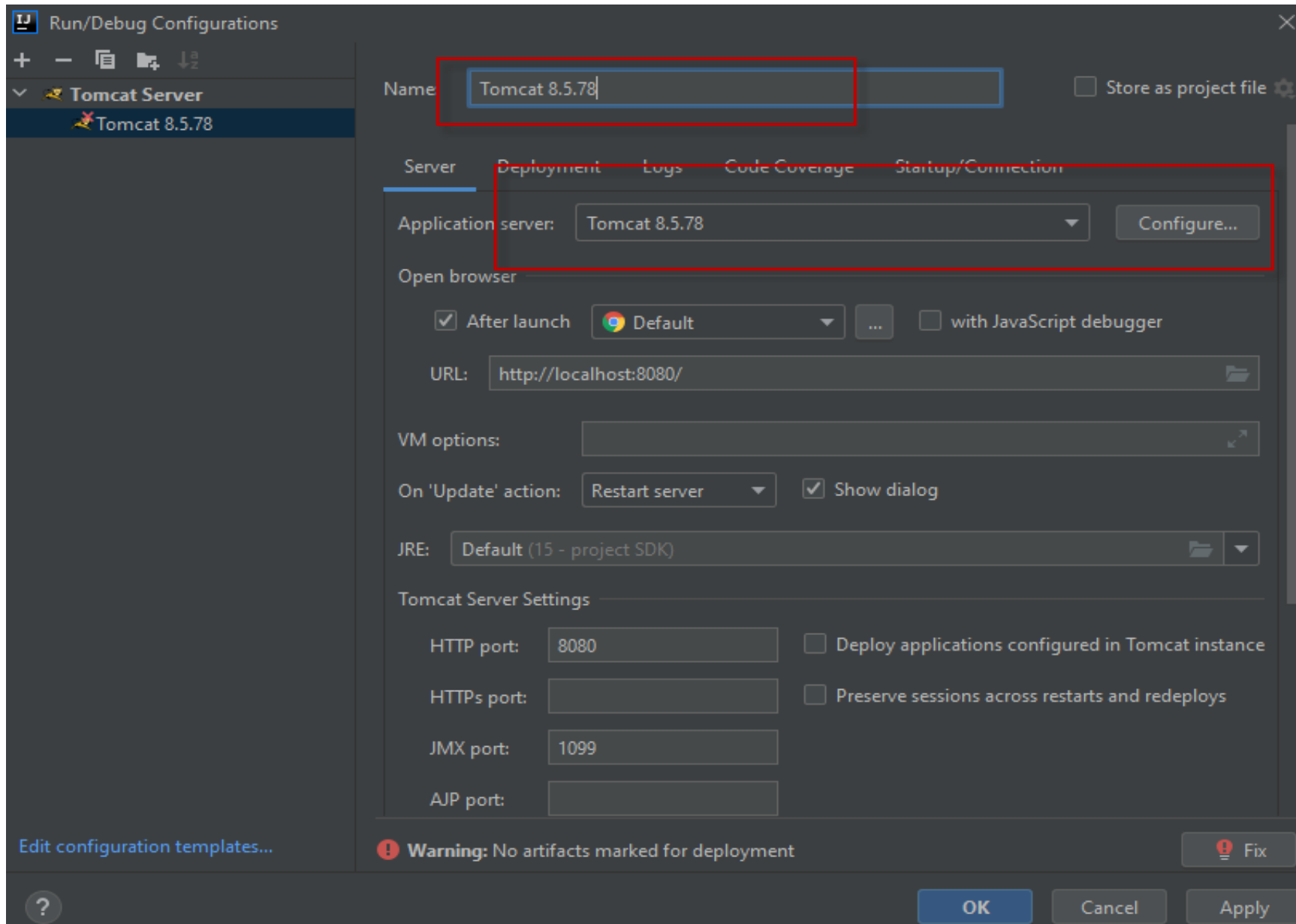


Cấu hình Web Server

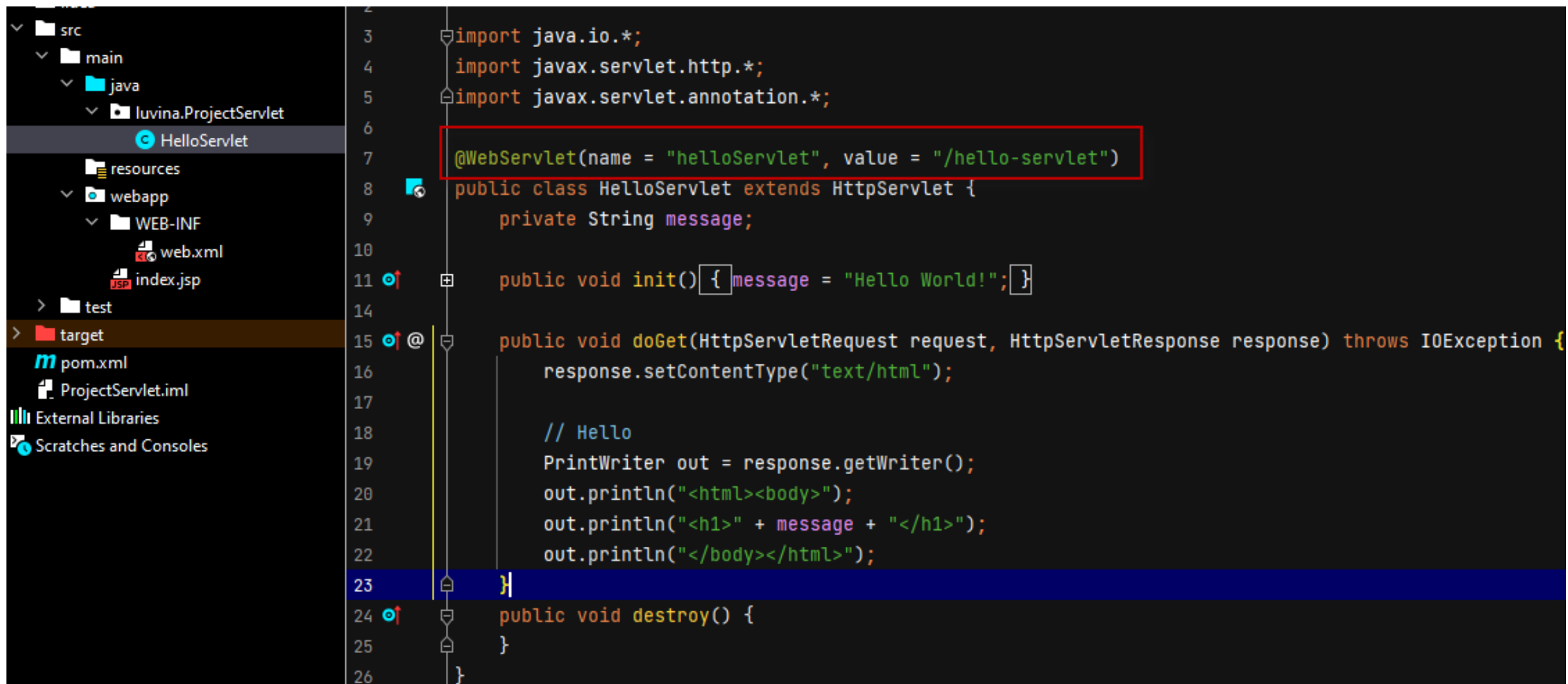


Cấu hình Web Server

15



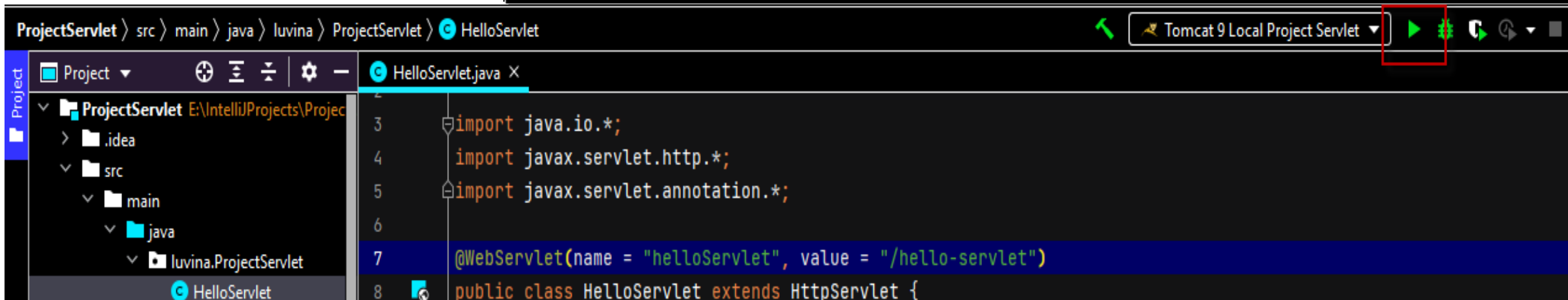
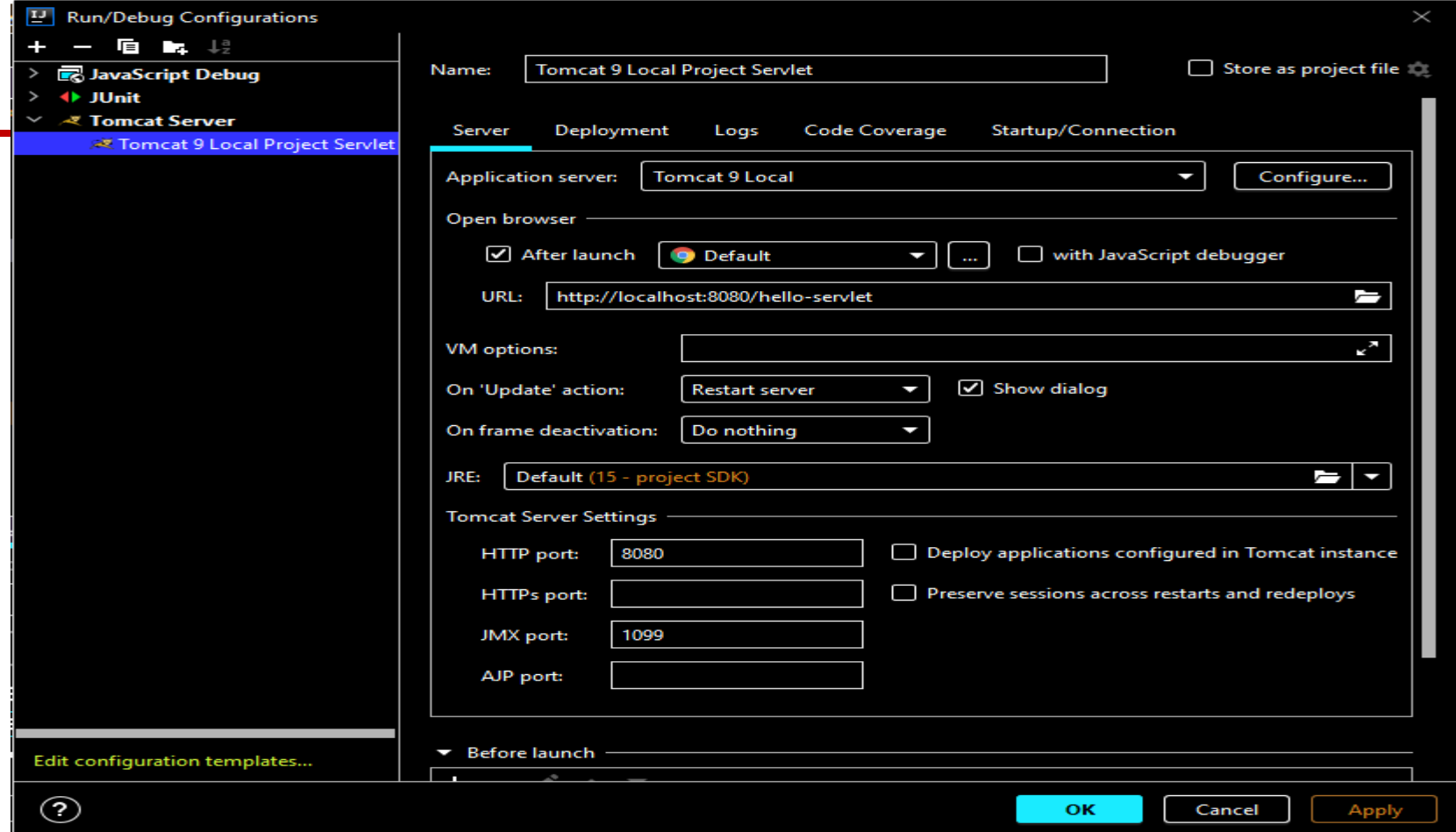
Cấu hình lớp servlet sử dụng annotation



The screenshot displays an IDE with a project structure on the left and a code editor on the right. The project structure shows a folder named 'src' containing a 'main' folder, which in turn contains a 'java' folder. Inside 'java', there is a package 'luvina.ProjectServlet' containing a class 'HelloServlet'. Below this, there is a 'resources' folder, a 'webapp' folder, and a 'WEB-INF' folder containing 'web.xml' and 'index.jsp'. The 'target' folder is also visible. The code editor shows the following code:

```
1  import java.io.*;
2
3  import javax.servlet.http.*;
4
5  import javax.servlet.annotation.*;
6
7  @WebServlet(name = "helloServlet", value = "/hello-servlet")
8  public class HelloServlet extends HttpServlet {
9      private String message;
10
11     public void init() { message = "Hello World!"; }
12
13
14
15     public void doGet(HttpServletRequest request, HttpServletResponse response) throws IOException {
16         response.setContentType("text/html");
17
18         // Hello
19         PrintWriter out = response.getWriter();
20         out.println("<html><body>");
21         out.println("<h1>" + message + "</h1>");
22         out.println("</body></html>");
23     }
24     public void destroy() {
25     }
26 }
```


Thực thi một Servlet



Giải pháp đối với vấn đề đa luồng

Thông thường, hệ thống **chỉ tạo ra một instance** ứng với servlet của bạn và sau đó mỗi khi có một request đến, hệ thống sẽ tạo ra một thread mới để xử lý request đó. Như vậy, nếu một request mới đến trong khi request trước vẫn đang thực thi thì sẽ xảy ra **nhiều thread** đồng thời đang truy cập vào **một object** của servlet.

Ví dụ, xét một servlet thực hiện gán giá trị **user ID** duy nhất cho mỗi client với việc sử dụng biến instance **nextID** như sau:

- ✓ `String id = "User-ID-" + nextID;`
- ✓ `out.println("<H2>" + id + "</H2>");`
- ✓ `nextID = nextID + 1;`
- Khi có nhiều luồng cùng chạy đoạn code trên thì sẽ dẫn đến có một số client nhận được cùng một giá trị ID.

Giải pháp đối với **vấn đề đa luồng**

1. Giải pháp 1: rút ngắn đoạn code trên
 - `String id = "User-ID-" + nextID++;`
2. Giải pháp 2 sử dụng giao diện **SingleThreadModel**.

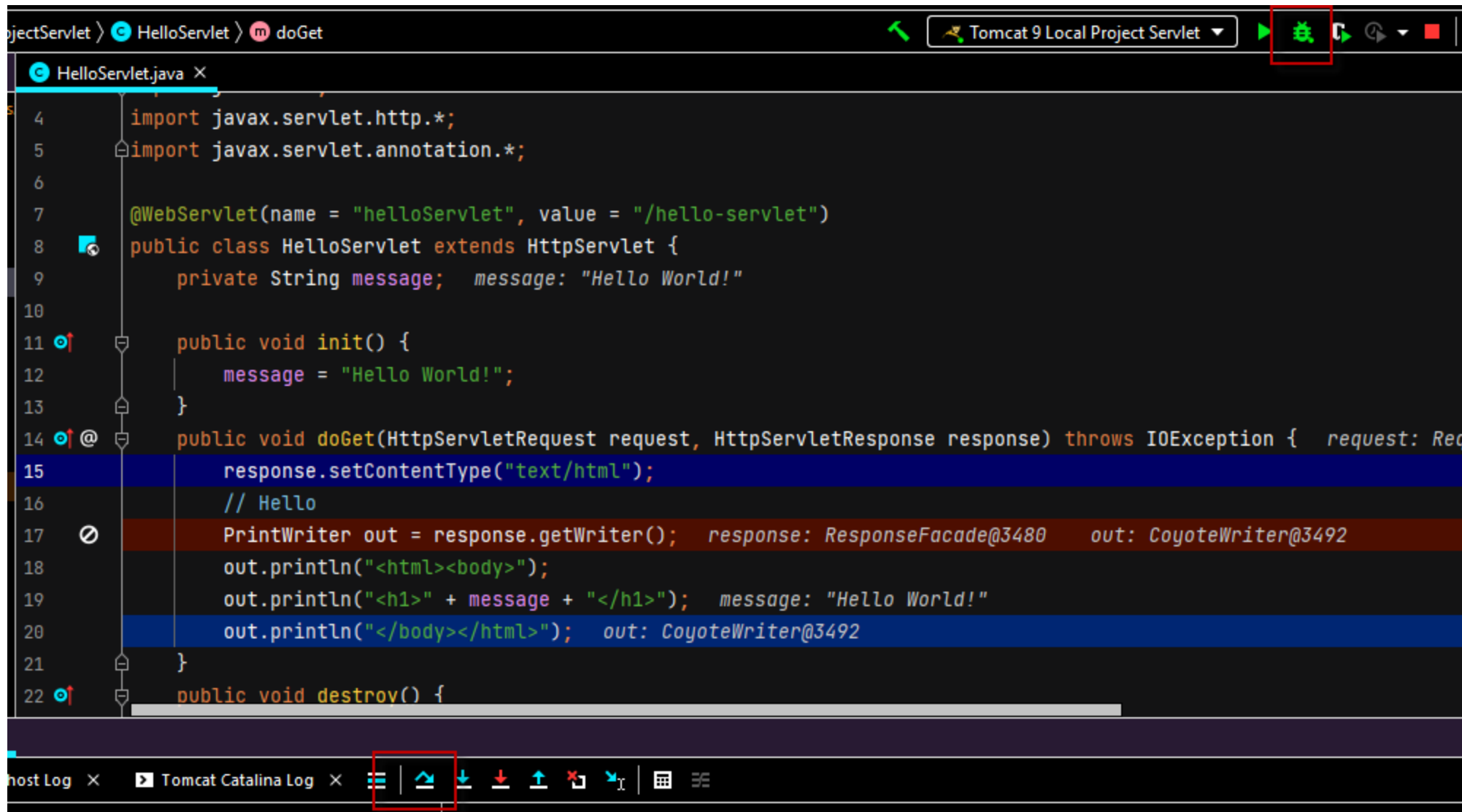
➤ `public class UserIDs extends HttpServlet`

`implements SingleThreadModel {}`

3. Giải pháp 3 sử dụng cấu trúc **synchronize** như sau:

```
synchronized (this) {  
    String id = "User-ID-" + nextID;  
    out.println("<H2>" + id + "</H2>");  
    nextID = nextID + 1;  
}
```

Giỡ rối chương trình Servlet



Đọc dữ liệu từ HTML Form

- Phương thức **getParameter** (tenControl): đọc dữ liệu đơn, tenControl phân biệt chữ hoa, thường:
 - `request.getParameter("Param1")` khác `request.getParameter("param1")`;
 - Tham khảo ví dụ trong Book1- 4.3 Example: Reading Three Parameters
 - Phương thức **getParameterValues** (tenControl): đọc nhiều giá trị. Nếu có nhiều control trùng tên được khai báo trên form thì `getParameterValues` (tenControl) sẽ trả về một mảng **xâu kí tự**.
 - Phương thức **getParameterNames** trả về danh sách tên tham số dưới dạng **Enumeration**
- Hoặc phương thức **getParameterMap** trả về một **Map**: có dạng key (tenControl) là tham số và value là giá trị của tham số (giá trị tương ứng của tenControl).

Ví dụ

Enumeration paramNames = **request.getParameterNames** ();

while (paramNames.hasMoreElements()) {

➤ String paramName = (String) paramNames.nextElement();

➤ out.print("<TR><TD>" + paramName + "\n<TD>");

➤ String[] paramValues = **request.getParameterValues** (paramName);

➤ if (paramValues.length == 1) {

String paramValue = paramValues[0];

if (paramValue.length() == 0) out.println("<I>No Value</I>");

else out.println(paramValue);

➤ } else {

out.println("");

for(int i=0; i<paramValues.length; i++) {

out.println("" + paramValues[i]);

}

out.println("");

➤ }

}

Ví dụ

A Sample FORM using POST

Item Number:

Description:

Price Each:

First Name:

Last Name:

Middle Initial:

Shipping Address:

Credit Card:

☐ Visa

☐ MasterCard

☐ American Express

☐ Discover

☒ Java SmartCard

Credit Card Number:

Repeat Credit Card Number:

Reading All Request Parameters

Parameter Name	Parameter Value(s)
cardNum	• 3.1415927 • 3.1415927
cardType	Java SmartCard
price	\$456.78
initial	No Value
itemNum	123-A
address	One Microsoft Way Redmond, WA 98052
description	Wild Wonder Widget
firstName	Sam
lastName	Palmisano

Tham khảo ví dụ trong Book1- 4.4
Example: Reading All Parameters

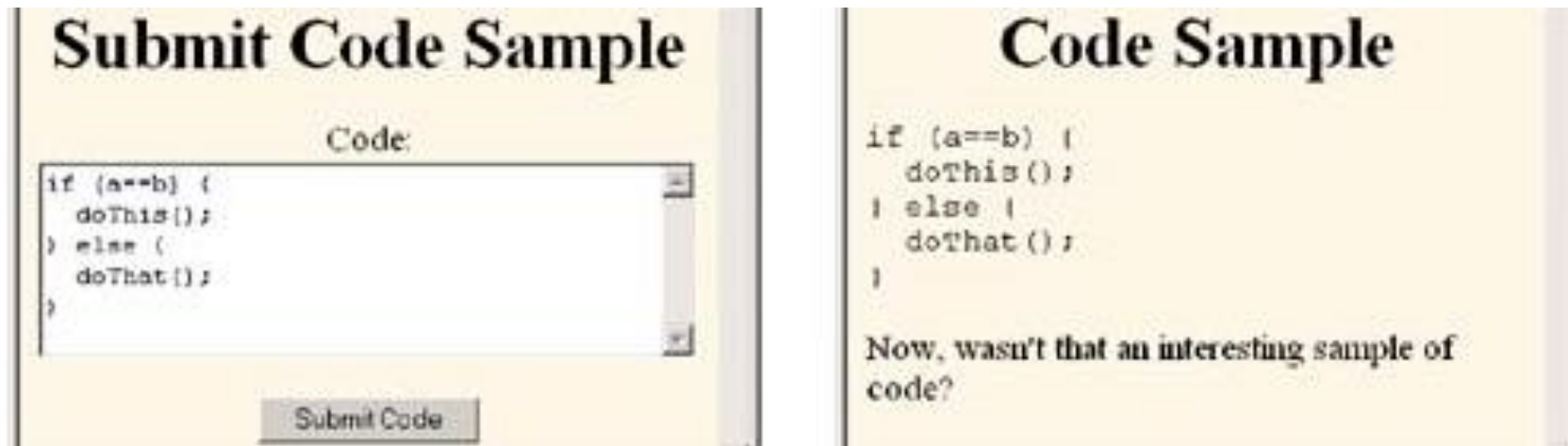
Sử dụng giá trị mặc định khi không đọc được dữ liệu từ control

```
String param = request.getParameter("someName");  
if ((param == null) || (param.trim().equals(""))) {  
    doSomethingForMissingValues(...);  
} else {  
    doSomethingWithParameter(param);  
}
```

UVINA

Lọc các ký tự đặc biệt

- **BadCodeServlet**: Kết quả vẫn tốt nếu như giá trị của tham số không chứa kí tự đặc biệt.

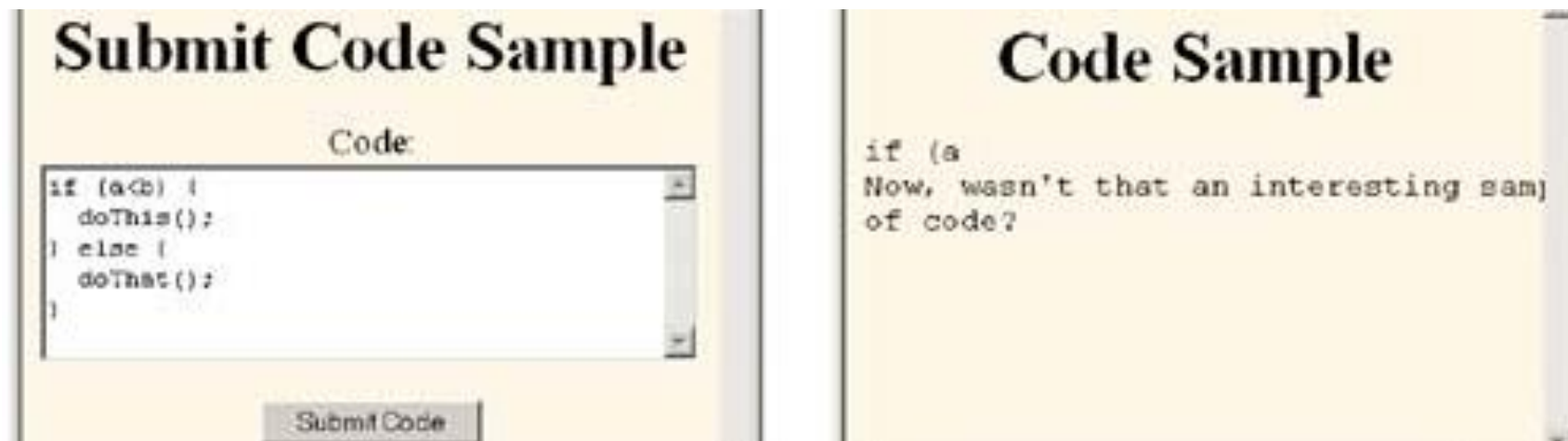


The screenshot shows two side-by-side panels. The left panel, titled "Submit Code Sample", contains a text area labeled "Code:" with the following Java code:

```
if (a==b) {  
    doThis();  
} else {  
    doThat();  
}
```

 Below the text area is a "Submit Code" button. The right panel, titled "Code Sample", displays the same code as entered, followed by the text: "Now, wasn't that an interesting sample of code?"

- **BadCodeServlet**: Kết quả sai



The screenshot shows two side-by-side panels. The left panel, titled "Submit Code Sample", contains a text area labeled "Code:" with the following Java code:

```
if (a<b) {  
    doThis();  
} else {  
    doThat();  
}
```

 Below the text area is a "Submit Code" button. The right panel, titled "Code Sample", displays the code as entered, but the output text is truncated: "if (a
Now, wasn't that an interesting sam
of code?"

Ví dụ: ServletUtilities

```
public class ServletUtilities {  
    public static String filter (String input) {  
        if (!hasSpecialChars(input)) { return (input); }  
  
        StringBuffer filtered = new StringBuffer(input.length());  
        char c;  
        for(int i=0; i<input.length(); i++) {  
            c = input.charAt(i);  
            switch(c) {  
                case '<': filtered.append("&lt;"); break;  
                case '>': filtered.append("&gt;"); break;  
                case '"': filtered.append("&quot;"); break;  
                case '&': filtered.append("&amp;"); break;  
                default: filtered.append(c); }  
        }  
        return(filtered.toString());  
    }  
}
```



Ví dụ: ServletUtilities

```
private static boolean hasSpecialChars (String input) {  
    boolean flag = false;  
    if ((input != null) && (input.length() > 0)) {  
        char c;  
        for(int i=0; i<input.length(); i++) {  
            c = input.charAt(i);  
            switch(c) {  
                case '<': flag = true; break;  
                case '>': flag = true; break;  
                case '\"': flag = true; break;  
                case '&': flag = true; break; }  
        }  
    }  
    return(flag);  
}}
```



Tham khảo ví dụ trong Book 1- 4.6 Filtering Strings for HTML-Specific Characters

Hiển thị lại Input form khi nhập thiếu giá trị

Tham khảo ví dụ trong Book 1- 4.8 Redisplaying the Input Form When Parameters Are Missing

**Welcome to Auctions-R-Us.
Please Enter Bid.**

It

Item Name:

Your Name:

Your Email Address:

Amount Bid:

Auto-increment bid to match other bidders?: ☐

**Welcome to Auctions-R-Us.
Please Enter Bid.**

Required Data Missing! Enter and Resubmit.

Item ID:

Item Name:

Your Name:

Required field! Your Email Address:

Amount Bid:

Auto-increment bid to match other bidders?: ☒

Bid Submitted

Your bid is now active. If your bid is successful, you will be notified within 24 hours of the close of bidding.

Willy's Wonder Widget
Item ID: 123-AB
Name: Marty Hall
Email address: hall@coreservlets.com
Bid price: \$456.78
Auto-increment price: true

Cộng tác giữa các Servlets

Giao diện **RequestDispatcher**

- **RequestDispatcher** cung cấp các phương thức **forward()** để điều hướng đến một tài nguyên khác (html, jsp, servlet);
- Cũng có thể sử dụng phương thức **include** để lấy nội dung của một tài nguyên khác và chèn vào trong servlet hiện thời.

Giao diện **HttpServletResponse**

- **sendRedirect()** method of **HttpServletResponse** interface điều hướng đến một tài nguyên khác (html, jsp, servlet);

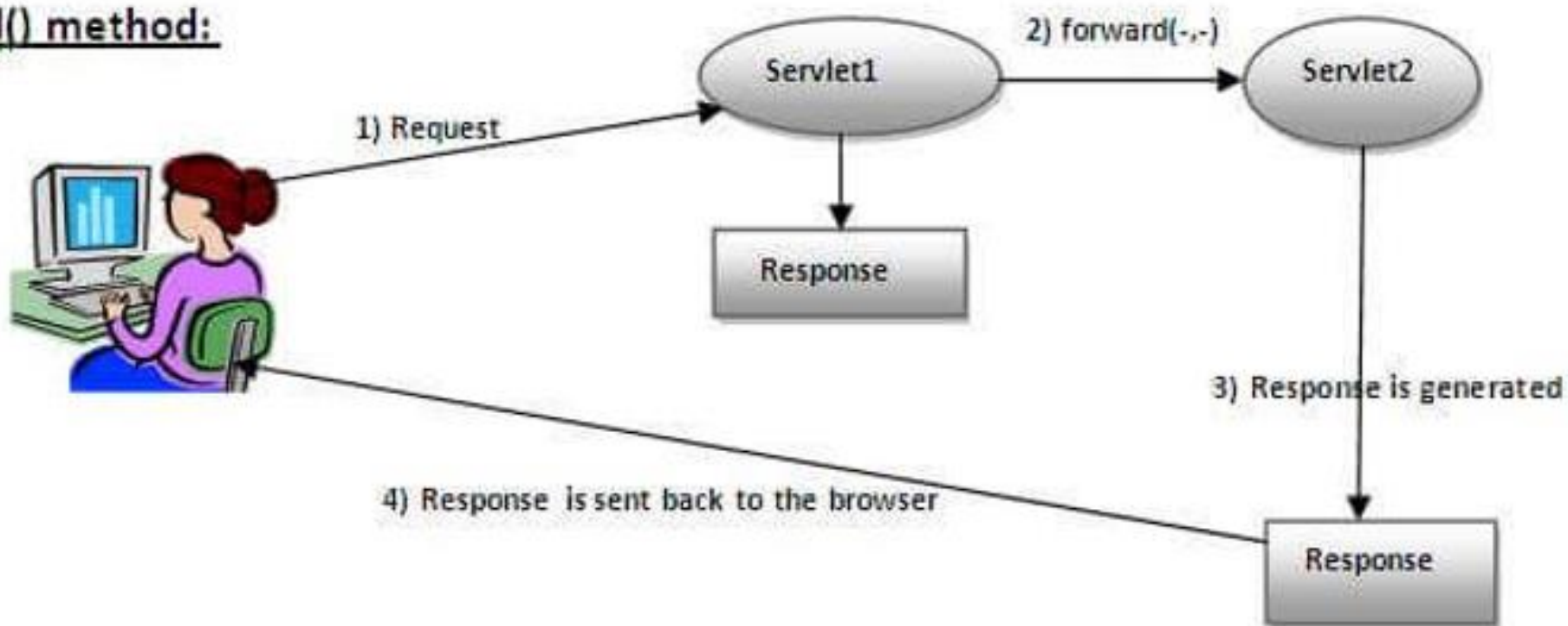
So sánh **RequestDispatcher** và **HttpServletResponse**

RequestDispatcher	HttpServletResponse
Servlet1: forward() - Làm việc ở trên server; - Gửi chính đối tượng request và response hiện thời đến tài nguyên khác; Ví dụ: <pre>request.getRequestDispatcher("servlet2") .forward(request, response);</pre>	Servlet1: sendRedirect() method - Làm việc ở client; - Gửi một request mới đến tài nguyên khác; Ví dụ: <pre>response.sendRedirect("servlet2");</pre>
Phương thức include() Để lấy nội dung của một tài nguyên khác và chèn vào trong servlet hiện thời.	Không có phương thức tương tự

Ví dụ về phương thức forward

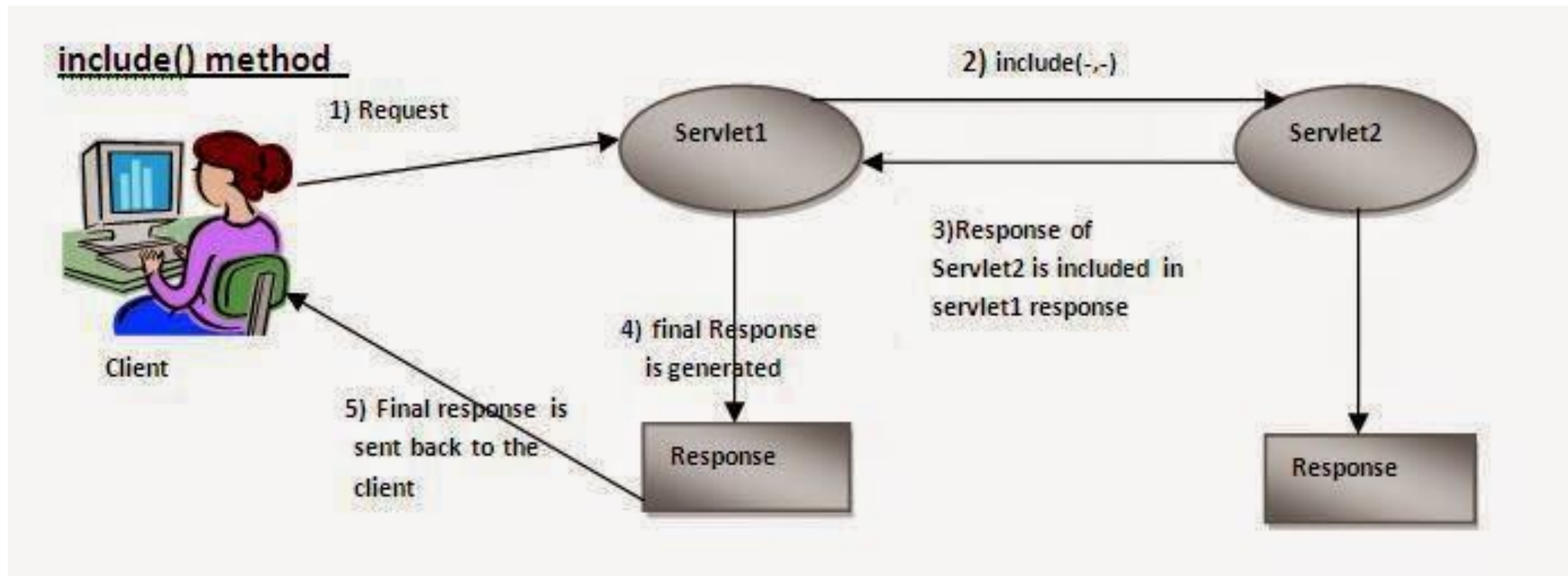
`public void forward (ServletRequest request,ServletResponse response) throws ServletException,java.io.IOException`

forward() method:

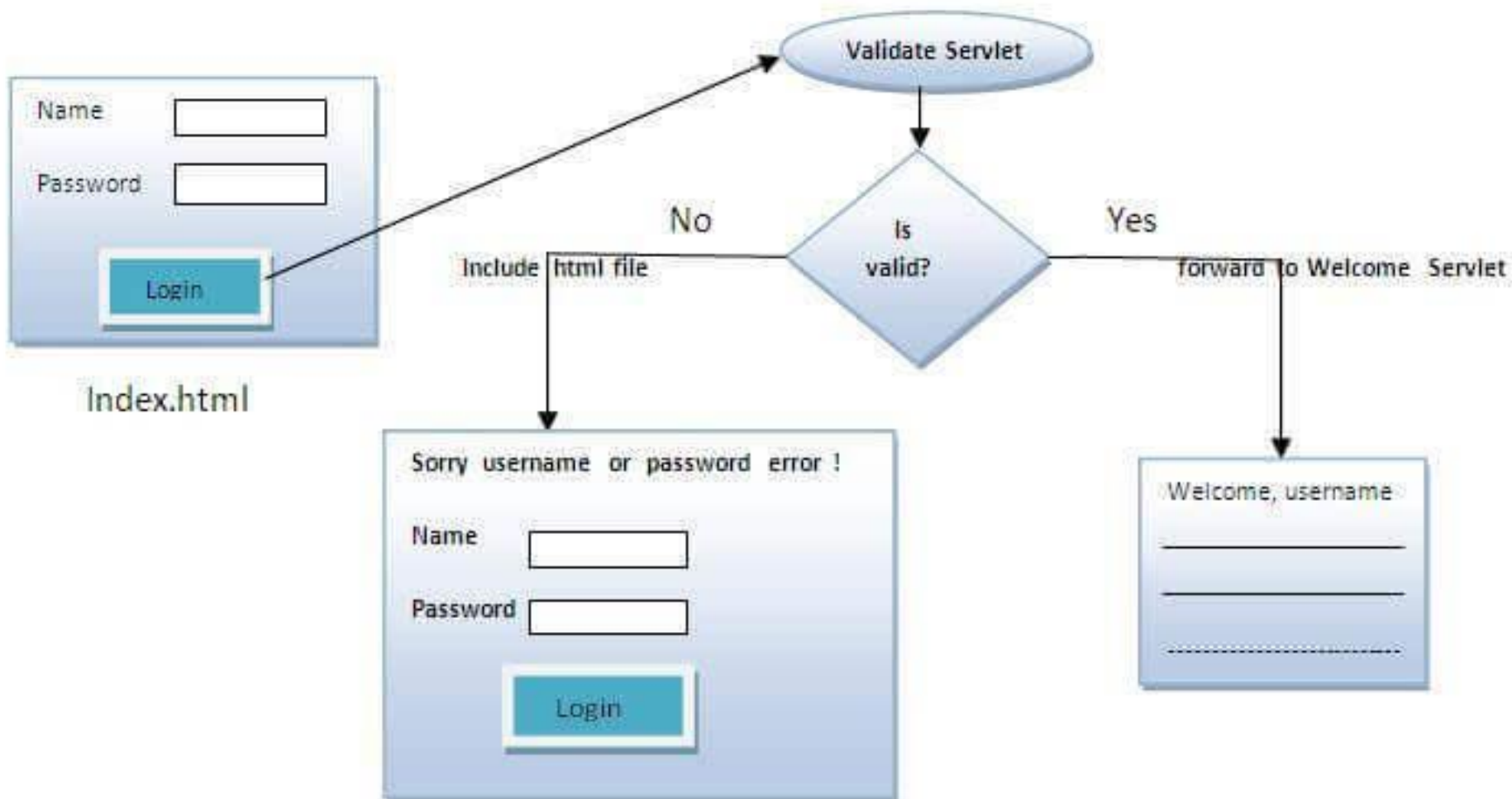


Ví dụ về phương thức include

`public void include (ServletRequest request,ServletResponse response)throws ServletException,java.io.IOException .`



Ví dụ về xử lý request (sơ đồ)

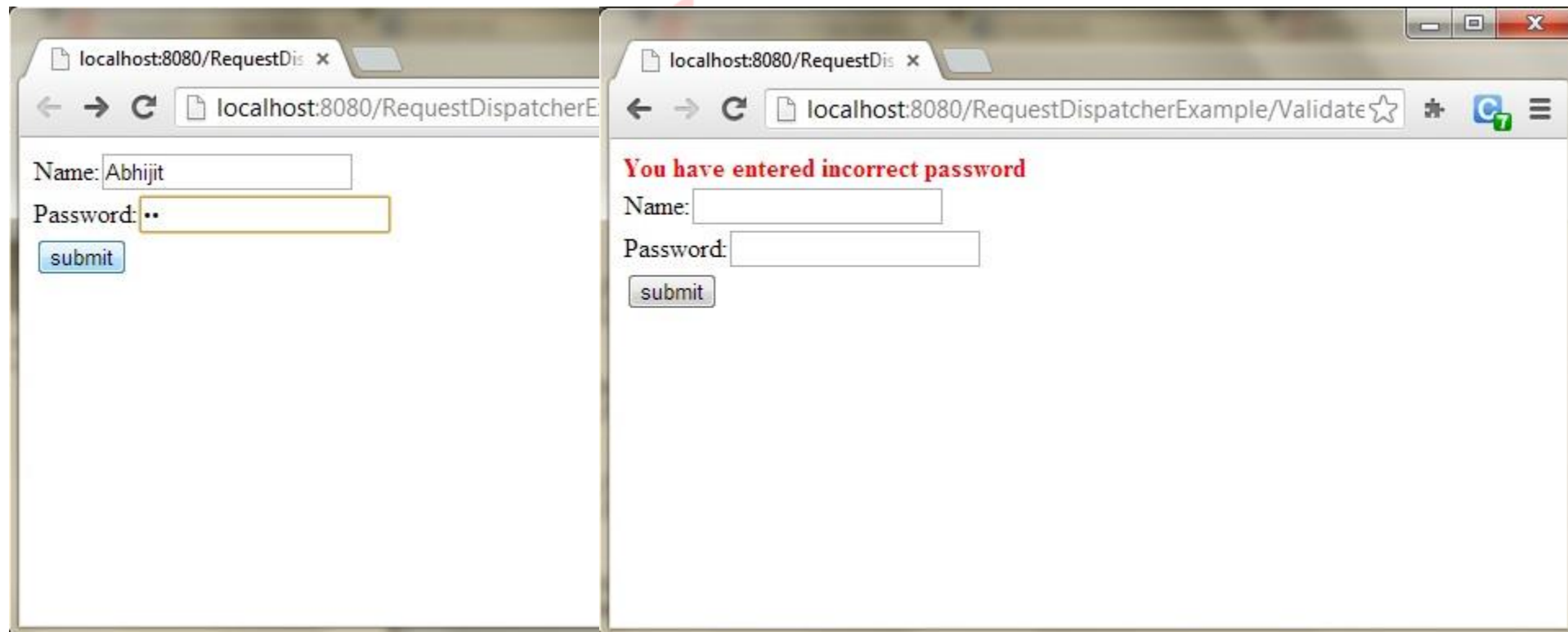


Ví dụ về xử lý request

Xem ví dụ:

- Index_dispatchRequest.html
- Packet dispatchRequest

Tham khảo thêm <https://www.javatpoint.com/requestdispatcher-in-servlet>



Ví dụ về phương thức sendRedirect

index.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>sendRedirect example</title>
</head>
<body>

<form action="MySearcher">
<input type="text" name="name">
<input type="submit" value="Google Search">
</form>

</body>
</html>
```

MySearcher.java

```
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class MySearcher extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String name=request.getParameter("name");
        response.sendRedirect("https://www.google.co.in/#q="+name);
    }
}
```

Tham khảo ví dụ tại:
[https://www.javatpoint.com/sendRedirect\(\)-method](https://www.javatpoint.com/sendRedirect()-method)

Các kỹ thuật lưu trữ

HTTP là một giao thức phi trạng thái, tức là các lệnh sẽ thực hiện độc lập với nhau. Điều này sẽ làm xuất hiện một số vấn đề đối với một vài hệ thống.

Cookies. được lưu trữ trên máy của bạn -> có thể làm xuất hiện một số vấn đề về mất an toàn bảo mật thông tin.

- Đối với Chrome thì cookies được lưu ở:
C:\Users\Your_User_Name\AppData\Local\Google\Chrome\User Data\Default\Network
- Đối với Microsoft Edge thì cookies được lưu ở thư mục:
%LOCALAPPDATA%\Microsoft\Edge\User Data\Default\Network

URL Rewriting. Chúng ta thêm một số thông tin vào cuối URL. Ví dụ:
http://localhost/adbc/indh?**data=xyz**

Hidden form fields. Thêm các trường ẩn vào Form

```
<INPUT TYPE="HIDDEN" NAME="session" VALUE="...">
```

Lưu trữ trong **Session** trên **server**

localStorage